601.433/633 Introduction to Algorithms **Fall 2026**
Homework #6 **Due:** October 29, 2026, 11:59pm

Remember: you may work in groups, but must write up your solution entirely on your own. Solutions must by typeset (LaTeX preferred but not required). Collaboration is limited to discussing the problems – you may not look at, compare, reuse, etc. any text from anyone else in the class. You may use the internet to look up formulas, definitions, etc., but may not simply look up the answers online.

Please include proofs with all of your answers, unless stated otherwise.

1 Computer Viruses (50 points)

You're helping some security analysts monitor a computer network, tracking the spread of an online virus. There are n computers in the system, labeled $C_1, C_2, \ldots, C_n$. You are given a collection of *trace data* indicating the times at which pairs of computers communicated. This data is a sequence of ordered triples (C_i, C_j, t_k), where such a triple indicates that C_i and C_j exchanged bits at time t_k. There are m triples total.

We'll assume that the triples are presented to you in sorted order of time, and for simplicity we'll assume that each pair of computers communicates at most once during the interval that you're observing.

The security analysts you're working with would like to able to answer questions of the following form: if the virus was inserted into computer C_a at time 0, what is the first time at which it could have have infected computer C_b? If it cannot be infected at all, then we say that this time is ∞.

The mechanics of infection are simple: if an infected computer C_i communicates with an uninfected computer C_j at time T_k (i.e., if one of the triples (C_i, C_j, t_k) or (C_j, C_i, t_k) appears in the trace data), then computer C_j becomes infected as well, starting at time t_k. Infection can thus spread from one machine to another across a *sequence* of communications, provided that no step in this sequence involves a move backward in time. Thus, for example, if C_i is infected by time t_k, and the trace data contains triples (C_i, C_j, t_k) and (C_j, C_q, t_r) where $t_k \leq t_r$, then C_q will become infected via C_j. Note that it is OK for t_k to be equal to t_r; this would mean that C_j had open connections to both C_i and C_q at the same time, and so a virus could move from C_i to C_q through C_j.

For example, suppose that $n = 4$ and the trace data consists of the triples

$$(C_1, C_2, 4), (C_2, C_4, 8), (C_3, C_4, 8), (C_1, C_4, 12)$$

and the virus was inserted into computer C_1 at time 0. Then C_3 would first be infected at time 8 by a series of three steps: first C_2 becomes infected at time 4, then C_4 from C_2 at time 8, and then C_3 from C_4 at time 8. So we should return 8 if the query is C_3.

On the other hand, if the trace data were

$$(C_2, C_3, 8), (C_1, C_4, 12), (C_1, C_2, 14)$$

and again the virus was inserted into C_1 at time 0, then C_3 would *not* become infected during the period of observation. So we should return ∞ if the query is C_3.

So your goal is to design an algorithm (as fast as possible) that answers questions of this type. The input is a collection of trace data (consisting of m triples), a virus injection point (a computer C_a), and another computer C_b. Your algorithm must return the first time at which C_b could be infected, assuming the virus was injected into C_a at time 0.

As always, prove correctness and running time.

Algorithm Description:

I use a weighted graph-based approach to solve this problem. We will use an undirected graph where the edges represent an interaction between 2 computers, and the edge weight will represent the timestamp. We represent this graph using an adjacency list, where each vertex is mapped to those it's adjacent to with a tuple of the vertex and the time. The creation of the graph will clearly take $O(n+m)$ time, since it will first initialize each list for the n computers (representing the vertices), and then iterate through the m tuples and add each interaction to the graph representation.

From here, we must determine the shortest path – that is, the fastest time it will take for C_b to be infected given C_a as the injection point. To do so, we will use a modifed version of Dijkstra's algorithm. Just like regular Dijksta, we maintain an array $d[n]$, representing the shortest distance from the source vertex, C_a. Initially, all values in the array are set to ∞. We maintain a minimum priority queue, initially containing C_a with priority 0.

Normally, in Dijksta, during the relaxation, the weights are added together to form the complete path. For vertex v communicating with vertex u, the shortest distance would be $d[v] = \min(d[v], d[u] + w(u,v))$. However, since the weight is the raw timestamp of communication, we can modify this: if $w(u,v) > d[u]$, then $d[v] = \min(d[v], \max(d[u], w(u,v)))$. This represents an infected computer u spreading the infection to computer v at a time after u has become infected.

The actual process of this algorithm would go as follows: Let v represent the vertex with the smallest tentative extraction time (root of the queue). First, v is extracted from the priority queue. Then, the algorithm examines each of its adjacent vertices, calculating the candidate infection time using the formula described above. If the candidate yields a smaller infection time, we update $d[v]$ and update the priority of v in the queue. This repeats until either the queue is empty or C_b is extracted. If C_b is extracted, we return $d[b]$, since its infection time has been finalized. Otherwise, we return ∞, since no valid infection path exists from C_a to C_b.

Correctness Proof:

This algorithm finds the first time at which C_b may have become infected. In order for an infection to occur along any path from C_a to C_b, we need the time stamps along that path to be nondecreasing. If this property holds, then the infection point of C_b is the timestamp on its communication with the last computer in the path. We are looking for the first possible infection point, so for all paths in the graph from C_a to C_b, we are looking for the path whose largest edge weight (representing the time at which C_b is infected) is as small as possible, with the constraint that the path weights are nondecreasing.

We will now prove that the modified version of Dijkstra's algorithm described above correctly finds this timestamp. Let $t(v)$ represent the earliest possible infection time of vertex v. We want to prove that when a vertex u is extracted from the priority queue, $d[u] = t(u)$.

We first establish that every finite tentative distance assigned by our algorithm represents a valid infection sequence. Initially, $d[a] = 0$, which is valid because C_a is infected at time 0. When we relax an arbitrary edge $u \leftrightarrow v$ with weight $w(u,v)$, we first check that $w(u,v) \geq d[u]$. This ensures that the communication occurs at or after the time at which u becomes infected. Therefore, if $d[u]$

represents a valid infection time, assigning $d[v] = w(u, v)$ also represents a valid infection time. Thus, every finite tentative distance represents a valid infection sequence, and thus $d[v] \geq t(v)$.

We now prove by induction on the order in which vertices are extracted from the priority queue that $d[u] = t(u)$ for every extracted vertex u.

Base case. The first vertex extracted is C_a, since $d[a] = 0$ and all other vertices have an initial distance of ∞. Since C_a is infected at time 0, we know that $d[a] = t(a) = 0$.

Inductive step. Assume that for every vertex x extracted before the current iteration, $d[x] = t(x)$. Let u be the next vertex extracted from the priority queue. We want to show that $d[u] = t(u)$. We already established that every finite tentative distance represents a valid infection sequence, so $d[u] \geq t(u)$. We therefore only need to show that $d[u] \leq t(u)$.

We assume FTSOC that $d[u] > t(u)$. We consider a valid infection sequence from C_a to u that infects u at time $t(u)$. Let j be the first vertex along this sequence that has not yet been extracted, and let i be the vertex immediately preceding j.

By the inductive hypothesis, $d[i] = t(i)$. Since the infection sequence is valid, the communication between i and j occurs at a time s such that $s \geq t(i) = d[i]$. Therefore, when i was extracted, the algorithm considered the edge $i \leftrightarrow j$ and assigned j a tentative distance $\leq s$.

Because the timestamps along the infection sequence are nondecreasing, $t \leq t(u)$. Thus, we have $d[j] \leq s \leq t(u) < d[u]$. However, this is a contradiction because the algorithm extracted u before j, even though j had a smaller tentative distance. The priority queue extracts the vertex with the smallest tentative distance. Therefore, the assumption must be false, and $d[u] = t(u)$.

By induction, every vertex extracted from the priority queue has its earliest possible infection time assigned to its distance. Thus, when C_b is extracted, $d[b]$ is the earliest possible time at which C_b becomes infected. If C_b is never extracted, no valid infection sequence exists from C_a to C_b, and the algorithm correctly returns ∞.

Therefore, we have proven the algorithm's correctness.

Runtime Proof:

There are 2 steps to this algorithm. The first is graph creation. Each of the n adjacency lists needs to be initialized, which takes $O(1)$ time each, meaning $O(n)$ for adjacency list initialization. Then, each of the m tuples must be inserted into as edges into the graph. This means that graph creation takes $O(n + m)$ time.

Now, we move on to the modified Dijkstra's algorithm. This follows the same process as normal Dijkstra's algorithm. The modified relaxation does not change the asymptotic number or cost of operations. Therefore, it has the same time complexity. As per the proof in Lecture Notes 15.4.1, we know that if we strucutre this with a Fibonacci heap, the time complexity of this algorithm is $O(m + n \log n)$. This amortized guarantee is enough.

Therefore, combining both steps, the total time complexity is $O(n + m) + O(m + n \log n)$, or $O(m + n \log n)$.

2 Consistent Lifetimes (50 points)

Suppose that you are an historian, and you are trying to figure out possible dates for when various historical figures may have lived. In particular, there are n people $P_1, P_2, \ldots, P_n$ who you are studying. Based on your research, you have discovered a collection of m facts about when these people lived relative to each other. Each of these facts has one of the following two forms:

- For some i and j, person P_i died before person P_j was born; or
- For some i and j, the lifetimes of P_i and P_j overlapped.

Unfortunately, since the historical record is never fully trustworthy, it's possible that some of these facts are incorrect. Design an $O(n + m)$ time algorithm to at least check whether they are *internally consistent*, i.e., whether there is a birth and a death date for each person so that all of the facts are true. As always, prove running time and correctness (i.e., prove that if there is a possible set of dates then your algorithm will return yes, and if no such dates are possible then your algorithm will return no).

Algorithm Description:

We use an unweighted directed graph-based algorithm. For each person P_i in the record, we create 2 vertices in the graph – B_i (their birth) and D_i (their death). This yields $2n$ vertices. We use directed edges to represent the chronology, so B_i has an edge directed towards D_i to represent the birth occurring before the death.

We construct a graph based on the given m facts. We initialize $2n$ adjacency lists to represent each known fact, along with inserting each edge $B_i \to D_i$ for each i, which is another n operations. Then, we iterate through the m facts. We will create the edges as follows for each fact:

(1) For some i and j, person P_i died before person P_j was born: $D_i \to B_j$.
(2) For some i and j, the lifetimes of P_i and P_j overlapped: $B_i \to D_j$ and $B_j \to D_i$.

This construction process takes $O(3n + m)$, or $O(n + m)$ time.

From here, we must detect directed cycles within the graph, since they show events that should take place after one another, which do not. We can do so using DFS with current path tracking. In this case, we keep track of 2 arrays along the DFS: $visited[]$ and $path[]$. In this algorithm, for each unvisited vertex, we initialize a DFS traversal. Each vertex we visit will be marked **true** in both arrays. We then recurse through all neighbors. If we encounter a node already marked **true** in $path[]$, a cycle is detected and the algorithm returns **no**. After exploring all of the neighbors of any vertex v, we backtrack and reset $path[v]$ to **false**. If we complete the traversal with no cycle detection, the algorithm returns **yes**.

Correctness Proof:

To prove correctness, we first prove that detecting cycles will detect any and all inconsistencies. Next, we will prove that this algorithm detects all cycles in a given record.

In this graph, every edge reflects a strict chronological relationship. This means an edge from arbitrary V_i and V_j implies that event V_i took place strictly before V_j. Thus, let us imagine there are arbitrary vertices V_x, V_y, and V_z. If there are edges from V_x and V_y, and V_y and V_z, this means that V_x took place strictly before V_z. Thus, if a cycle forms, among these 3 vertices, this means there exists an edge from V_z to V_x, which is impossible within the internal logic of the graph. Therefore, we can see that a cycle is indicative of an inconsistency in the logic. We can generalize this argument to any arbitrary directed cycle, as that indicates that some event took place after an event is has already been established to take place before.

When there are no cycles, this makes the graph a directed acyclic graph by design. Thus, there exists some topological ordering of the vertices such that for every arbitrary directed edge $V_i \to V_j$, V_i appears before V_j. Since we have included an edge representing every fact in the dataset, in

establishing a topological ordering, we know that all of the original facts must be satisfied. Thus, in establishing there are no cycles, we would prove the data is logically consistent.

Finally, we consider **Theorem 14.2.1** in the lecture notes: A directed graph G is a DAG if and only if DFS(G) has no back edges. In our algorithm, the *path*[] array tracks back edges. Therefore, in our algorithm finding if back edges exist via DFS, we prove that the graph constructed is a directed acyclic graph, and therefore, it is logically consistent.

Runtime Proof:

There are 2 main steps to this algorithm, the creation of the graph and running DFS.

In initializing the graph as an adjacency list, we will first iterate through the $2n$ nodes, and we will create an array for each node to track edges. From here, for each of the n people P_i, we will insert edge $B_i \to D_i$. We can see that this is another n operations. From here, we will iterate through each of the m facts and insert 1 to 2 edges per fact. Therefore, we insert at most $2m$ edges in this step. Therefore, the worst case runtime for graph creation is $O(2n) + O(n + 2m)$, or $O(3n + 2m)$. Simplifying, this yields $O(n + m)$.

After this, we will run DFS on the resulting graph. By the lecture notes, we know that we spend $\Theta(V)$ time initializing the DFS-tracking arrays *visited*[] and *path*[], where V is the number of vertices. There are $2n$ vertices, so this takes $\Theta(n)$ time. From here, since we visit each vertex only once, we only scan each adjacency list once. Thus, we spend $O(E)$ time, were E is the number of edges. As per the initialization, we know there are $O(n + m)$ edges, so this step takes $O(n + m)$ time. Thus, this part of the algorithm takes $O(n) + O(n + m)$ time, or $O(n + m)$.

Since the graph creation and DFS run in sequence, the total runtime is the sum of their respective runtimes, so this algorithm runs in $O(n + m) + O(n + m)$ time, or $O(n + m)$ time.